

# UNIT 1: FUNDAMENTALS OF SEARCHING AND CLASSICAL METHODS

Internal Searches, Classical Methods, Complexity Analysis

Author: Eng. Carlos Andrés Sierra, M.Sc.  
cavirguezs@udistrital.edu.co

Full-time Adjunct Professor  
Computer Engineering Program  
School of Engineering  
Universidad Distrital Francisco José de Caldas

2026-II



# Course Outline

1 Fundamental Concepts

2 Classical Methods



# Outline

## 1 Fundamental Concepts

## 2 Classical Methods



# Core Terminology

## Searching

The algorithmic process of **locating** a specific **target** record within a collection of data or an underlying data structure.

- **Tinder**
  - **Google**
  - **Chat GPT**
- Record: A complete data entity containing multiple fields (e.g., a ...)
- Key: The distinct identifier used by an algorithm to perform lookups.
- Key Field: The specific class attribute or field targeted during comparison steps.

Search Key Target

|                |                   |
|----------------|-------------------|
| ID (Key Field) | Name, Email, Role |
|----------------|-------------------|

Complete Record Object

*list tree graph*



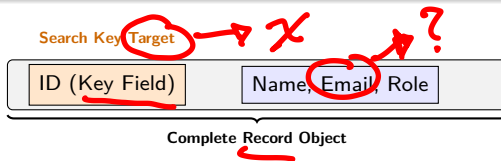
# Core Terminology

## Searching

The algorithmic process of **locating a specific target record** within a collection of data or an underlying data structure.

## Data Anatomy

- **Record**: A complete data entity containing multiple fields (e.g., a User struct). *cc, code, id*
- **Key**: The distinct identifier used by an algorithm to perform lookups. *age + name*
- **Key Field**: The specific class attribute or field targeted during comparison steps.



# Internal vs. External Searches

## Internal Search

Executed entirely within **Main Memory (RAM)**

- Fast random-access times.
- Limited by physical volatile capacity. *~ 5 Tb*
- Low operational access cost.

## External Search

Executed across **Secondary Storage (Disk/ies)**

- Suitable to massive non-volatile files.
- Constrained by high disk latency.
- Optimization focuses on reducing block reads.

*int list = [ ]*

*list[27890]*

*memory*



# Internal vs. External Searches

## Internal Search

Executed entirely within **Main Memory (RAM)**.

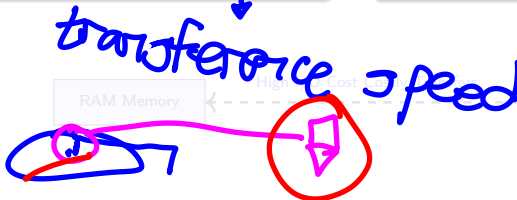
- Fast random-access times.
- Limited by physical volatile capacity.
- Low operational access cost.

## External Search

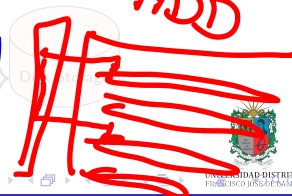
Executed across **Secondary Storage (Disks/Files)**.

- Scalable to massive **non-volatile** files.
- **Constrained** by high **I/O disk latency**.
- Optimization focuses on reducing disk block reads.

SSD



HDD



# Internal vs. External Searches

## Internal Search

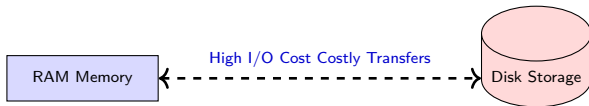
Executed entirely within **Main Memory (RAM)**.

- Fast random-access times.
- Limited by physical volatile capacity.
- Low operational access cost.

## External Search

Executed across **Secondary Storage** (Disks/Files).

- Scalable to massive non-volatile files.
- Constrained by high I/O disk latency.
- Optimization focuses on reducing disk block reads.



# Performance Measures & Complexity

## Algorithmic Evaluation Criteria

To evaluate a **search method's performance**, we observe *three* metrics:

- ❶ **Number of Comparisons:** Primary index calculation factor.
- ❷ **Execution Time:** Absolute CPU cycle time requirements.
- ❸ **Disk Access Operations:** Crucial for external block I/O analysis.

## Asymptotic Analysis (Big-O)

We classify searching algorithms based on how their execution scaling factors behave relative to a dataset size  $n$  as  $n \rightarrow \infty$ .



# Performance Measures & Complexity

## Algorithmic Evaluation Criteria

To evaluate a **search method's performance**, we observe *three* metrics:

- ❶ **Number of Comparisons:** Primary index calculation factor.
- ❷ **Execution Time:** Absolute CPU cycle time requirements.
- ❸ **Disk Access Operations:** Crucial for external block I/O analysis.

## Asymptotic Analysis (Big-O)

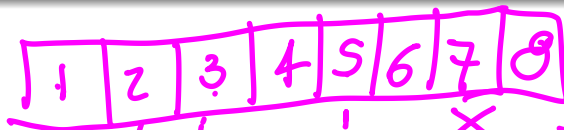
We classify searching algorithms based on how their execution scaling factors behave relative to a dataset size  $n$  as  $n \rightarrow \infty$ .



# Outline

1 Fundamental Concepts

2 Classical Methods



$2^3$   
 $\log(17)$

$\text{int}(\text{mid})$

$\text{int}(a.s)$

0

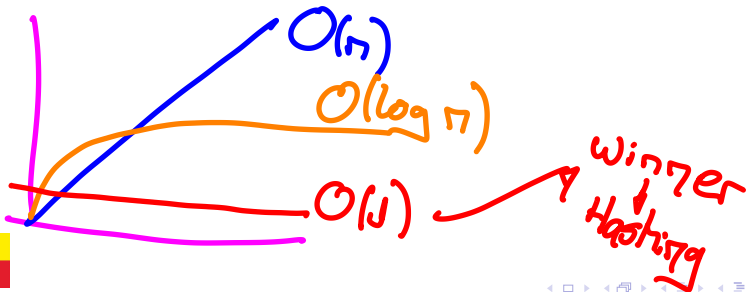


# Classical Methods

The **classical methods** are a set of foundational search algorithms that have been widely studied and applied in computer science. They include:

- Sequential Search (Linear Search)
- Binary Search
- Key Transformation Search (Hashing)

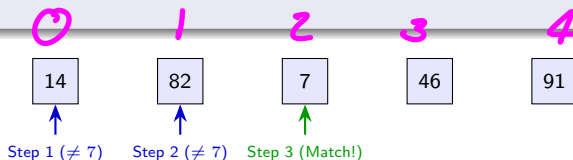
Each method has its own unique approach to searching through data, with varying performance characteristics and use cases.



# Sequential Search

## Behavioral Model

Linearly processes a collection from index 0 to  $n - 1$ . Suitable for **unsorted** datasets.



## Complexity Analysis:

- Best Case:  $O(1)$   
Element at index 0
- Worst Case:  $O(n)$   
Element at end/missing
- Average Case:  $O(n)$

**Key Characteristic:** No ordering prerequisites; highly efficient for small arrays, completely unsuitable for large collections.

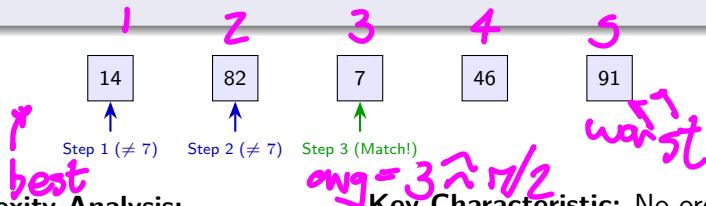
*easy to implement*



# Sequential Search

## Behavioral Model

Linearly processes a collection from index 0 to  $n - 1$ . Suitable for **unsorted** datasets.



## Complexity Analysis:

- **Best Case:**  $O(1)$   
Element at index 0
- **Worst Case:**  $O(n)$   
Element at end/missing
- **Average Case:**  $O(n)$

**Key Characteristic:** No ordering prerequisites; highly efficient for small arrays, completely unscalable for large collections.

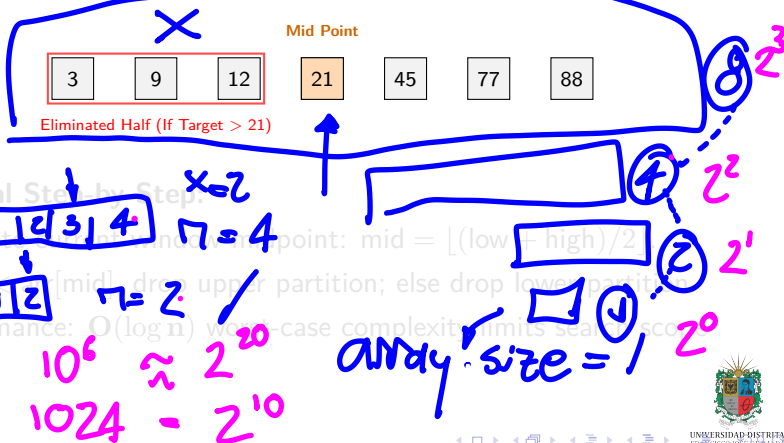
$$O\left(\frac{n}{2}\right) = O(n)$$



# Binary Search

## Prerequisite Requirement

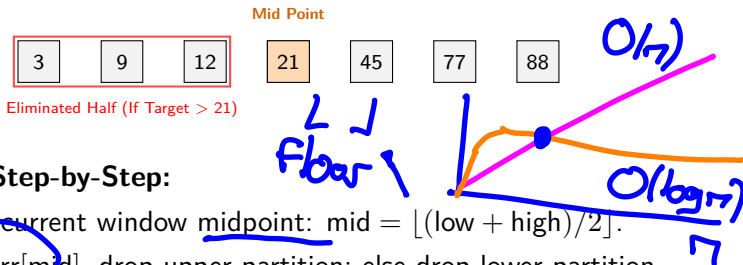
The target dataset must be sorted. Uses a divide-and-conquer strategy to cut the search space in half at each iteration.



# Binary Search

## Prerequisite Requirement

The target dataset **must be sorted**. Uses a divide-and-conquer strategy to cut the search space in half at each iteration.



## Operational Step-by-Step:

- Compute current window midpoint:  $\text{mid} = \lfloor (\text{low} + \text{high}) / 2 \rfloor$ .
- If  $\text{key} < \text{arr}[\text{mid}]$ , drop upper partition; else drop lower partition.
- Performance:  $O(\log n)$  worst-case complexity limits search scope rapidly.

Quicksort  
Merge sort

}

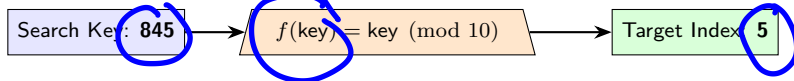
Binary Search



# Key Transformation Search

## Direct Index Computation

Bypasses structural step traversals by **maps**. The **key** itself is run through a **mathematical mapping function** to compute its position array index directly.



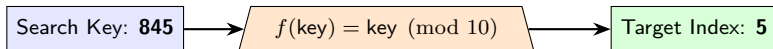
- Serves as the conceptual foundation for Hashing.
  - Indexed Mapping Target Complexity:  $O(1)$ .
  - Operational Challenge: Multiple independent keys mapping to matching slots triggers index assignment conflicts.
- Handwritten notes:*
- A red 'x' is drawn over the first bullet point.
  - A red arrow labeled 'map' points from the first bullet point to a red circle containing a plus sign and a minus sign.
  - A pink arrow points from the text 'data representation' to the word 'computation'.



# Key Transformation Search

## Direct Index Computation

Bypasses structural step traversals by **maps**. The **key** itself is run through a **mathematical mapping function** to compute its position array index directly.



- Serves as the conceptual foundation for **Hashing**.
- Idealized Mapping Target Complexity:  $O(1)$ .
- Operational Challenge: Multiple independent keys mapping to matching slots triggers index assignment conflicts.



# Classical Methods Comparison

| Search Method      | Best Case | Worst Case    | Prerequisites     |
|--------------------|-----------|---------------|-------------------|
| Sequential Search  | $O(1)$ ✓  | $O(n)$ ✓      | None Required     |
| Binary Search      | $O(1)$ ✓  | $O(\log n)$ ✓ | Sorted Collection |
| Key Transformation | $O(1)$ ✓  | $O(1)^*$ ✓    | Mapping Function  |

\* Assumes ideal distribution with zero address generation overlaps.

## Key Takeaways

Use **sequential search** for short, dynamic arrays.

Switch to **binary search** for large, stable datasets.

Implement **key transformations** to achieve optimal search times when memory allocation headroom allows.



# Classical Methods Comparison

| Search Method      | Best Case | Worst Case  | Prerequisites     |
|--------------------|-----------|-------------|-------------------|
| Sequential Search  | $O(1)$    | $O(n)$      | None Required     |
| Binary Search      | $O(1)$    | $O(\log n)$ | Sorted Collection |
| Key Transformation | $O(1)$    | $O(1)^*$    | Mapping Function  |

\* Assumes ideal distribution with zero address generation overlaps.

## Key Takeaways

Use **sequential search** for short, dynamic arrays.

Switch to **binary search** for large, stable datasets.

Implement **key transformations** to achieve optimal search times when memory allocation headroom allows.

no frequent  
sort & unsort

complex



# Outline



#(v) #protesta  
#votaciones

$f(\text{key})$

protesta -  - 

cd - 

votaciones - 

esma - 



#protesta #canal



# Thanks!

## Questions?



Repo: <https://github.com/EngAndres/ud-public/tree/main/courses/computer-sciences-ii>

